

Matrix as Plan: Structured Logical Reasoning with Feedback-Driven Replanning

Ke Chen*
kechen@mail.bnu.edu.cn
Faculty of Arts and Sciences
Beijing Normal University
Zhuhai, Guangdong, China

Jiandian Zeng*
jiandian@bnu.edu.cn
Institute of Artificial Intelligence and
Future Networks
Beijing Normal University
Zhuhai, Guangdong, China

Zihao Peng
pzh_cs@mail.bnu.edu.cn
Institute of Artificial Intelligence and
Future Networks
Beijing Normal University
Zhuhai, Guangdong, China

Guo Li
liguo@mail.bnu.edu.cn
Institute of Artificial Intelligence and
Future Networks
Beijing Normal University
Zhuhai, Guangdong, China

Guangxue Zhang
guangxue_zhang@bnu.edu.cn
Institute of Artificial Intelligence and
Future Networks
Beijing Normal University
Zhuhai, Guangdong, China

Tian Wang†
tianwang@bnu.edu.cn
Engineering Research Center of
Cloud-Edge Intelligent Collaboration
on Big Data, Ministry of Education
Beijing Normal University
Zhuhai, Guangdong, China

Abstract

As knowledge and semantics on the web grow increasingly complex, enhancing Large Language Models (LLMs)' comprehension and reasoning capabilities has become particularly important. Chain-of-Thought (CoT) prompting has been shown to enhance the reasoning capabilities of LLMs. However, it still falls short on logical reasoning tasks that rely on symbolic expressions and strict deductive rules. Neuro-symbolic methods address this gap by enforcing formal correctness through external solvers. Yet these solvers are highly format-sensitive, and small instabilities in model outputs can lead to frequent processing failures. The LLM-driven approaches avoid parsing brittleness, but they lack structured representations and process-level error-correction mechanisms. To further enhance the logical reasoning capabilities of LLMs, we propose MatrixCoT, a structured CoT framework with a matrix-based plan. Specifically, we normalize and type natural language expressions and attach explicit citation fields, and introduce a matrix-based planning method to preserve global relations among steps. The plan thus becomes a verifiable artifact and execution becomes more stable. For verification, we also add a feedback-driven replanning mechanism. Under semantic-equivalence constraints, it identifies omissions and defects, rewrites and compresses the dependency matrix, and produces a more trustworthy final answer. Experiments on five logical-reasoning benchmarks and five LLMs show that, without relying on external solvers, MatrixCoT enhances both the robustness and interpretability of LLMs when tackling complex symbolic reasoning tasks, while maintaining competitive performance.

*Equal contribution.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792213>

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; **Natural language processing**;

Keywords

Logical Reasoning, Large Language Models, Neurosymbolic Approaches, Semantic Decomposition

ACM Reference Format:

Ke Chen, Jiandian Zeng, Zihao Peng, Guo Li, Guangxue Zhang, and Tian Wang. 2026. Matrix as Plan: Structured Logical Reasoning with Feedback-Driven Replanning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792213>

1 Introduction

The Web's explosive growth has created a vast, heterogeneous substrate of knowledge. Extracting task-relevant content and reliable semantic knowledge from these repositories is essential [11, 18, 19]. This necessity has motivated the integration of Large Language Models (LLMs) into web-based applications, including browsing-based question answering, cross-site web agents, and LLM-assisted information extraction and knowledge-graph construction [3, 15, 30, 35]. Despite their strong capabilities in natural language generation, text classification, and machine translation [1, 4, 10], current LLMs remain unreliable on complex logical reasoning [20, 23]. The gap is most evident in higher-order tasks such as mathematical problem solving and intricate logical inference. Consequently, to ensure trustworthy behavior in web applications, developing robust reasoning is essential for advancing LLMs within the Web's semantics and knowledge ecosystem.

Motivated by this need, a growing body of work now examines how LLMs perform on such reasoning tasks. Current LLMs are pretrained primarily on human-written natural text, where the corpora lack high-quality examples of logical reasoning, deduction, and proof [5, 14, 27]. Moreover, prevailing training objectives, such

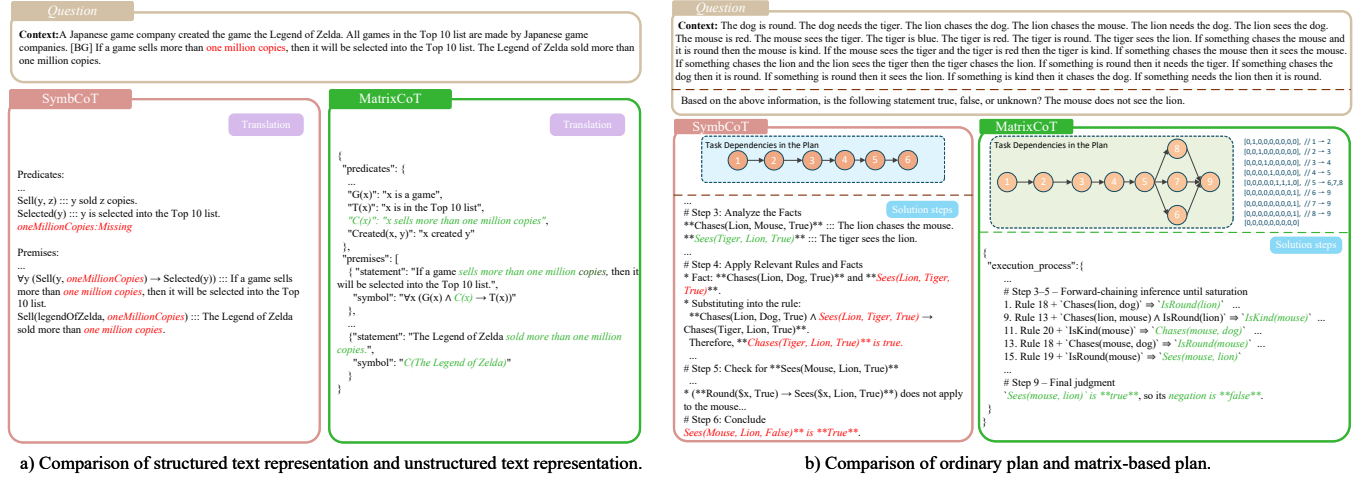


Figure 1: Illustration of Logical Reasoning with SymbCoT and the Proposed MatrixCoT. a) Structured vs. unstructured text representation. b) Ordinary plan vs. matrix-based plan.

as next token prediction or masked language modeling, bias models toward learning syntax, semantics, and commonsense knowledge [13]. As a result, even state-of-the-art LLMs still struggle to achieve satisfactory performance on complex logical reasoning tasks. To improve the reasoning capacity, researchers have proposed Chain-of-Thought (CoT) prompting, which encourages step-by-step decomposition of complex problems [28]. Variants such as Tree of Thoughts (ToT) [33] and Graph of Thoughts (GoT) [2] further extend this idea. While these methods enhance reasoning to some degree, they remain limited by the abstraction and vagueness of natural language. Genuine logical reasoning, however, requires symbolic representations and formal deduction rules, which are largely absent from textual corpora. The tension is especially pronounced in settings that demand rigorous logical representations and formal derivations.

To address the above limitations, researchers have explored several neuro-symbolic approaches. One prominent line of work emphasizes a hybrid architecture that integrates an LLM with a symbolic reasoning backend [6, 21]: 1) The LLM first translates a natural language statement of the reasoning problem into a set of first-order logic formulas; and 2) a symbolic solver uses this formal representation to automatically plan the reasoning steps, execute the derivations, and produce the final answer. Within this pipeline, the symbolic solver plays a pivotal role in both planning and execution. It imposes constraints and performs verification to mitigate reasoning errors that arise from the inherent abstraction and vagueness of natural language formulations [8, 16, 17, 34].

However, external reasoners require extremely strict formatting on their inputs. Even slight deviations introduced by an LLM during generation can cause parsing to fail and the entire reasoning pipeline to halt. To fill the gap, LLM-driven reasoning methods [12, 31, 32] are designed to be robust to syntactic noise and structural perturbations. A representative example is SymbCoT [32]: rather than relying on any symbolic solver, the LLM is adopted in a systematic four-stage pipeline: translation, planning, solving, and verification, suggesting that LLM-driven reasoning frameworks

offer strong generative flexibility. However, in practice, they exhibit limitations that may degrade reasoning performance:

- Reasoning by LLMs on prose-style text intermixes predicates, entities, and premises within free-form narration.** It maintains only loose alignment through inline annotations (e.g., ::). In the absence of a unified namespace and field-level constraints, types and variable scope can not be statically checked, and cross-sentential references are prone to drift. As shown in Fig. 1(a), in an unstructured rendering without a unified namespace, an expression such as `Sell(y, oneMillionCopies)` inserts `oneMillionCopies` as a bare constant in the second argument. This token is neither defined in the predicate inventory nor annotated with a type or a unit. Consequently, the rule $\forall y (\text{Sell}(y, \text{oneMillionCopies}) \rightarrow \text{Selected}(y))$ can only be treated passively as a “special-case match,” rather than expressing the computable semantics $\text{sales} \geq 10^6$. Signature constraints and static checks therefore cannot be enforced. The system cannot promptly or reliably detect and constrain such semantic and typing omissions, nor can it proactively require completion. Subsequent reasoning then runs on incomplete dependencies, making unstable or erroneous outcomes likely. By contrast, structured symbolic translation, under a unified namespace, uses keys and indices for precise addressing. It can perform static checks and consistency validation for type consistency, variable scope, and cross-sentential references. With equivalent informational content, it thus yields higher decidability and overall accuracy.
- Linearized planning struggles to explicitly encode and schedule inter-step dependencies.** It is sensitive to wording and sampling perturbations, which often break reasoning chains and cause evidence to be missed. Take Fig. 1(b) as an example. The correct shortest reasoning chain is: `Chases(lion, dog, True) → Round(lion, True)`; then, combined with `Chases(lion, mouse, True)`, derive `Kind(mouse, True)`; next, `Kind(mouse, True) → Chases(mouse, dog, True) → Round(mouse, True) → Sees(mouse, lion, True)`. Therefore, `not(Sees(mouse, lion, True))` is False. However, the executed linear plan exhibits “local inference not closed”

and “semantic drift.” At Step 4, it applies the rules only once. It then proceeds to the derivability test in Step 5 without forward-chaining to a fixed point. As a result, the knowledge base lacks key facts that should have been entailed—Kind(mouse, True), Chases(mouse, dog, True), and Round(mouse, True). Consequently, it misclassifies the query as “underivable.” It also confuses the argument direction in rule premises, conflating Sees(lion, tiger, True) with the given Sees(tiger, lion, True), which further amplifies the error. In contrast, a matrix-based plan renders the cross-step dependency structure explicit. Reasoning paths are constrained by an adjacency matrix. Expansion of the logical chain is enforced in dependency order. It prevents skipping and omission and more systematically detects cyclic inconsistencies. Consequently, the correct answer could be obtained.

To tackle the above issues, we propose MatrixCoT, a structured CoT approach that employs a matrix-based planning method for robust logical reasoning. The goal is to systematically enhance the reliability and controllability of LLMs on logical reasoning tasks. Unlike traditional implementations that rely on linear chains and natural language narration, MatrixCoT retains a LLM-driven paradigm while introducing structured symbolic representations, a matrix-based planning method, and a feedback-driven replanning mechanism. Specifically, we organize reasoning data into a structured symbolic form where predicates, entities, rules, and questions are encoded with explicit types and links. The reasoning plan is then represented as a dependency matrix A , whose rows and columns denote steps and whose entries capture prerequisite–successor relations. It turns the plan from a fragile text-based script into a globally consistent, computable structure that avoids ambiguity and preserves inter-step relations. Finally, a feedback-driven replanner examines solver traces to identify issues such as missing premises or conflicts. With minimal edits, A is updated and the instance is re-solved, thereby strengthening robustness and accuracy. Our contributions can be summarized as follows:

- **MatrixCoT.** This paper proposes MatrixCoT, a structured CoT method for logical reasoning. Compared with existing approaches, MatrixCoT organizes LLM outputs into structured representations and adopts matrix-based planning with feedback-driven replanning to guide execution more clearly, thereby achieving better robustness and interpretability on logical reasoning tasks.
- **Structured data management.** We introduce a method that converts natural language expressions into a normalized, typed representation with explicit referential links. This makes logical dependencies explicit, curbs semantic ambiguity, and improves both execution stability and answer quality.
- **Matrix-based planning method.** In this method, we represent the reasoning plan as a unified dependency matrix. This matrix preserves the global relations among steps, thereby turning the plan into a verifiable, computable artifact and further improving the consistency and controllability of the LLM reasoning process.
- **Feedback-driven replanning mechanism.** The system uses previous solution trajectories and candidate answers as audit signals. Under semantic equivalence constraints, omissions and defects are identified. The dependency matrix

is then rewritten and compressed, thereby producing a more trustworthy final answer.

2 Methodology

As presented in Fig. 2, the MatrixCoT framework comprises four core modules: a symbol translator, a matrix-based planner, a problem solver, and a feedback-driven replanner. In this section, the structured data management, a matrix-based planning method, and a feedback-driven replanning mechanism are detailed.

2.1 Structured Data Management

Recent studies show that converting the reasoning process into structured representations can alleviate the issues above [29]. Structured carriers reduce semantic ambiguity and explicitly maintain dependencies between pieces of information. In CoT settings, embedding intermediate reasoning steps into parseable fields significantly improves the accuracy of mathematical and symbolic reasoning. Some reports even claim that adding just a single field to carry the reasoning yields about a 60% gain on GSM8K [9]. Going further, the schema design itself directly affects performance [26]. Small changes in field names or ordering can cause orders-of-magnitude differences; for example, renaming “final_choice” to “answer” can raise accuracy from 4.5% to 95% [9].

Inspired by the above research, we propose a prompt-driven structured representation that covers the full pipeline of symbolic reasoning. For illustration, in the symbol translation process, we define a parsing map from a natural language sequence $x_{1:T}$ (a sequence of text spans of length T) to symbolic objects:

$$f_{\text{prompt}} : x_{1:T} \longrightarrow S = \langle \mathcal{V}, \mathcal{K}, \mathcal{F}, \mathcal{R}, \mathcal{Q} \rangle. \quad (1)$$

Here, $\mathcal{V}$ denotes the symbolic vocabulary; $\mathcal{K}$ the set of constants; $\mathcal{F}$ the set of normalized facts; $\mathcal{R}$ the set of rules; and $\mathcal{Q}$ the set of propositions to be adjudicated. At the symbolic layer we adopt a standard first-order logical vocabulary, for example the usual Boolean connectives, quantifiers, and equality, such that:

$$\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \forall, \exists, =\} \subseteq \mathcal{L}_{\text{FO}}. \quad (2)$$

Finally, we serialize S into a key–value form $\text{enc}(S)$, defined as an alignment between Natural Language (NL) and symbolic expressions:

$$\text{enc}(S) = \{(NL_i, \text{Symbol}_i) \mid i = 1, \dots, N\}. \quad (3)$$

This general mapping captures the correspondence between each natural language statement and its canonical symbolic form, facilitating downstream auditing and reasoning. Compared with free-text representations that maintain only a loosely aligned structure via inline markers (e.g., “:::”), this representation reduces semantic ambiguity and the propagation of redundancy. It also provides stable anchors for planners and solvers. In feedback-driven settings, it further supports auditable intermediate evidence and replanning. Without sacrificing expressivity, this structured representation improves the reliability and controllability of complex symbolic reasoning.

2.2 Matrix-based planning method

Most contemporary reasoning-augmentation methods treat free-form text as the primary organizational unit. CoT develops the

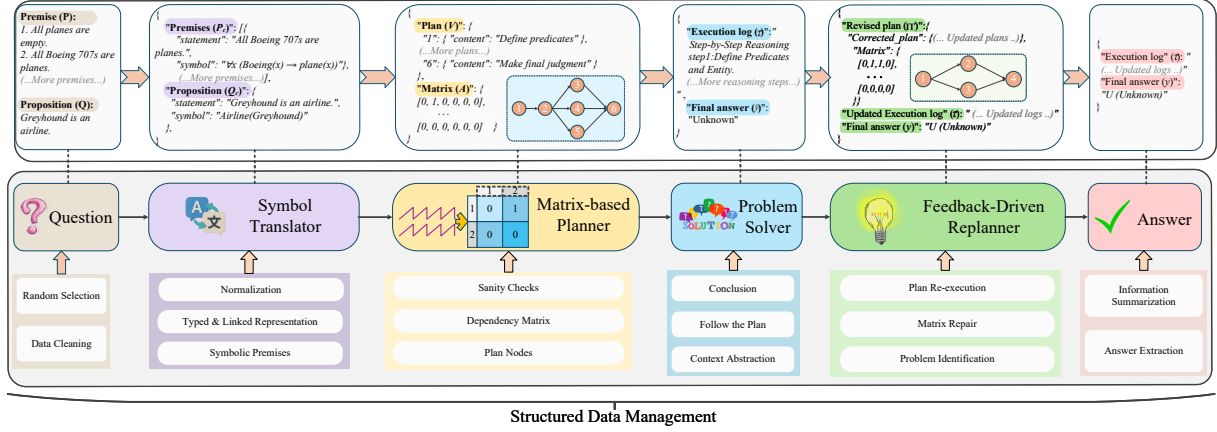


Figure 2: An overview of the MatrixCoT framework and methodology. The pipeline first translates the question into normalized, typed symbolic representations, then builds a matrix-based plan with dependencies and plan nodes. A problem solver executes the plan next, followed by a feedback-driven replanner to repair and re-execute the plan when needed. The system ultimately summarizes the information and extracts the answer.

derivation through step-by-step narration. Self-Consistency stabilizes outcomes via voting across multiple chains. Least-to-Most first decomposes and then recomposes the problem. ToT seeks to orchestrate “thought units” within a tree. Representative pipeline implementations of “plan-then-solve” also fall into this family. These approaches improve performance to a degree. Yet the “plan” is still largely stitched together from textual fragments. Prerequisite–successor relations across steps are maintained mainly by implicit ordering or heuristic scores, which makes it difficult to accumulate an explicit, machine-checkable dependency structure. When backtracking, pruning, or re-execution is required, plans across samples or iterations lack strong alignment and comparability. Search-style expansions additionally introduce substantial computational overhead and instability.

Building on these observations, we introduce a matrix-based planning method. It specifies reasoning steps and their precedence constraints directly and in a computable way. Formally, the plan is written as $\Pi = \langle S, A \rangle$: here $S = \{s_1, \dots, s_N\}$ is a finite set of steps. Each s_i denotes an inference step. N is the number of steps. The matrix $A \in \{0, 1\}^{N \times N}$ is a binary precedence matrix that encodes dependencies:

$$A_{ij} = 1 \Rightarrow s_i \prec s_j, \quad (4)$$

$$A_{ij} = 0 \Rightarrow s_i \parallel s_j. \quad (5)$$

Here, $\prec$ means “directly precedes.” The symbol $\parallel$ means “there is no direct dependency edge between the two.” The row index i marks a “source step,” and the column index j marks a “target step.” Therefore, a column corresponds to the set of prerequisites required before execution, and a row corresponds to the set of successors that are unlocked after completion.

To further clarify the semantics, we define two set operations:

$$\text{pred}(s_j) = \{s_i \in S \mid A_{ij} = 1\}, \quad (6)$$

$$\text{succ}(s_i) = \{s_j \in S \mid A_{ij} = 1\}. \quad (7)$$

Here, $\text{pred}(s_j)$ is the full set of predecessors of step s_j , which must be finished before executing s_j . $\text{succ}(s_i)$ is the full set of successors of step s_i , which can be directly triggered once s_i is done. If column j is all zeros, then $\text{pred}(s_j) = \emptyset$, denoting that the step has no prerequisites. If row i is all zeros, then $\text{succ}(s_i) = \emptyset$, signifying that the step has no successors.

Based on the above, the execution of the plan can also be formalized. At any time, select all steps s_j satisfying $\text{pred}(s_j) \subseteq S_{\text{done}}$ into the executable set, where $S_{\text{done}} \subseteq S$ is the set of completed steps. Then execute these steps and merge them into S_{done} . Repeat this process until all steps are completed or no new steps can be triggered. The execution procedure follows the structure of the matrix A directly. It avoids inferring dependencies from textual descriptions and ensures clarity, robustness, and controllability of the reasoning flow.

2.3 Feedback-Driven Replanning Mechanism

In complex symbolic reasoning, relying solely on free-form text or a one-shot linear plan often buries key dependencies in the narrative. Premises go unused; precedence relations are obscured by the prose; evidence becomes “unreachable” along long chains. Even retries tend to fall into the same traps. We have already used a structured semantic representation to stably index predicates, entities, and rules. We also encode “which step precedes which” as an explicit dependency matrix A via a matrix-based plan, so the execution side no longer has to guess dependencies. Nevertheless, an offline-generated A can still be misaligned with the true difficulties of execution. Evidence produced by one step may not be consumed in time (a broken chain). Some conclusions may be consumed too early (poor timing). Parallel branches may drift out of sync, causing repeated idle cycles. A good static plan is not the same as good dynamic execution. What is missing is a channel that feeds problems exposed during execution back into the plan’s structure.

To this end, we propose a feedback-driven replanning mechanism. It diagnoses and repairs historical execution traces. Let the

context be C , the original plan $\Pi = (V, A)$ (where V is the set of steps, typically $1, \dots, N$; each element corresponds to a planned reasoning step; $A \in 0, 1^{N \times N}$ is the dependency matrix), the execution log τ , and the provisional decisions D . The execution trace produced by a single solve is denoted:

$$\tau = \langle C, \Pi, T \rangle. \quad (8)$$

Using the joint evidence from the trace and the decisions, we define a diagnostic mapping:

$$f_{\text{diag}} : (\tau, D) \longrightarrow \mathcal{E}, \quad (9)$$

where $\mathcal{E}$ is a set of failure labels. It covers categories such as “missing prerequisites,” “rule misuse,” “premature termination,” and “redundancy.” In this way, diagnosis is converted from plain text to a structured object $(V, A, \mathcal{E})$, thus providing auditable anchors for subsequent plan editing and verification.

After issues are detected, the system performs structured edits using the dependency matrix A as the unified carrier. Let $\mathcal{U}$ be the set of edit operators (e.g., $\text{AddEdge}(i, j)$, $\text{DelEdge}(i, j)$, $\text{Merge}(p, q)$, $\text{InsertGuard}(k)$, etc.). The repair is expressed as:

$$\Pi' = (V', A') = \text{Apply}(\mathcal{U}, (V, A)), \quad (10)$$

$$\Pi'' = \text{Normalize}(\Pi'), \quad (11)$$

where Normalize removes redundancy, breaks cycles, and ensures a valid topological ordering. Re-execution depends only on “the prior context + the currently revised plan.” The process is simple and reproducible:

$$\text{Exec}(C, \Pi'') \longrightarrow (y, T'), \quad (12)$$

where y is the new answer and T' is the new log. Π'' fills in the necessary prerequisites and removes spurious or transitively redundant dependencies. Normalization ensures that the main chain executes completely in topological order. With these guarantees, re-execution avoids biases such as “premature stop,” “missing reasoning,” and “dependency errors.” Thus, without changing the input context C , it yields more accurate and robust answers for complex logical reasoning.

3 Logical Symbolic Reasoning

Task Definition. Given a set of premises $P = \{p_1, \dots, p_n\}$ (each p_i is a logical sentence) and a proposition Q to be decided. The possible answer is true (T), false (F), or unknown (U). Specifically, T means Q can be logically derived from P ; F means $\neg Q$ can be logically derived from P ; U means that, given P , the truth value of Q cannot be determined.

► **Example:**
 <Premises> (P)
 All cats are mammals. Tom is a mammal.
 <Proposition> (Q)
 Tom is a cat.
 <Answer>
 Unknown.

3.1 Step 1: Translating

The Translator maps natural language context to a structured symbolic representation. Specifically, given the raw premises $P = \{p_1, \dots, p_n\}$ and a proposition Q to be decided, the Translator

first produces the corresponding first-order logical forms $P' = \{p'_1, \dots, p'_n\}$ and Q' . Here, each p'_i and Q' is a well-formed formula over the standard first-order vocabulary (e.g., $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \forall, \exists, =\} \subseteq \mathcal{L}_{\text{FO}}$). It then constructs a hybrid representation for downstream reasoning: for the premises, it creates the aligned set $P_c = \{\langle p_i, p'_i \rangle\}_{i=1}^n$; for the proposition, it forms $Q_c = \langle Q, Q' \rangle$. Finally, these components are combined into a structured symbolic representation $\mathcal{H} = \{P_c, Q_c\}$ for use in subsequent processing.

► **Input:**

Please convert the context and question into formulas in First-Order Logic.

<Premises> (P)

Fiona is quiet.

(... More premises ...)

<Proposition> (Q)

Charlie is kind.

► **Output:**

<Structured symbolic representation> ($\mathcal{H}$)

```
{
  "Premises ( $P_c$ )": [
    {
      "statement": "Humans are mammals.",
      "symbol": " $\forall x (\text{Human}(x) \rightarrow \text{Mammal}(x))$ "
    },
    (... More premises ...)
  ],
  "Proposition ( $Q_c$ )": { "statement": "There is an animal.", "symbol": " $\exists x A(x)$ " }
}
```

3.2 Step 2: Matrix-based Planner

From the structured representation $\mathcal{H} = \{P_c, Q_c\}$ given by the symbol translator, a matrix-based planner constructs an executable dependency plan $\Pi = (V, A)$. Here, $V = \{v_1, \dots, v_m\}$ is a finite set of reasoning steps. $A = [A_{ij}]_{i,j=1}^m \in \{0, 1\}^{m \times m}$ is a dependency matrix. For each entry A_{ij} , if $A_{ij} = 1$, then $v_i \prec v_j$, i.e., v_i must be executed before v_j . The graph formed by A is required to be a directed acyclic graph, which ensures the existence of a topological order to obtain a valid execution schedule and avoids execution difficulties caused by interleaved dependencies. Accordingly, we obtain a dependency plan Π that is aligned with P_c and Q_c and has a clear structure, thereby balancing dependency and executability without sacrificing expressive power.

► **Input:**

Please derive a step-by-step plan using First-Order Logic rules and a

matrix-based method to determine the conclusion from the context.

<Structured symbolic representation> ($\mathcal{H}$)

► **Output:**

<Matrix-based Plan> (Π)

```
{
  "Plan ( $V$ )": {
    "1": { "content": "Define predicates." },
    "2": { "content": "Define constants: Monkeypox" },
    (... More plans ...)
    "11": { "content": "Make final judgment: True" }
  },
  "Matrix ( $A$ )": {
    [0,1,1,1,1,1,1,0,0,0],
    [0,0,1,1,1,1,1,0,0,0],
    ...
    [0,0,0,0,0,0,0,0,0,0]
  }
}
```

3.3 Step 3: Solver

Given the symbolized premises $P_c = \{\langle p_i, p'_i \rangle\}_{i=1}^n$, the proposition $Q_c = \langle Q, Q' \rangle$, and the matrix-based plan $\Pi = (V, A)$, the **Solver** executes the steps by following a linear order consistent with the dependency relations in the plan Π . Concretely, the Solver refers to the premises P_c and performs inference according to the plan Π , whose structure guarantees a valid topological execution order. After completing all steps, it uses the overall information of the reasoning process Γ . Then it makes a three-valued decision on Q_c :

$$\hat{y} = \begin{cases} \text{T,} & \text{if } \Gamma \vdash Q_c, \\ \text{F,} & \text{if } \Gamma \vdash \neg Q_c, \\ \text{U,} & \text{if } \Gamma \not\vdash Q_c \text{ and } \Gamma \not\vdash \neg Q_c. \end{cases} \quad (13)$$

The final output is the label $\hat{y} \in \{\text{T, F, U}\}$ together with execution log τ aligned with the dependency plan.

► **Input:**
Please solve the question based on the context and a matrix-based plan.
<Structured symbolic representation> ($\mathcal{H} = \{P_c, Q_c\}$)
<Matrix-based Plan> (Π)
► **Output:**
{
 "Execution log (τ)": "Analyze steps according to the dependency matrix...",
 "Final answer ($\hat{y}$)": {"T (True)"}
}

3.4 Step 4: Feedback-driven replanner

The Feedback-driven replanner treats the trace τ and the provisional outcome $\hat{y}$ as diagnostic evidence. The replanner inspects the dependency plan $\Pi = (V, A)$ and identifies structural inconsistencies. It then applies minimal dependency edits that preserve semantic validity and ensure topological executability. This procedure yields a revised plan $\Pi' = (V, A')$. A revision rationale accompanies the plan and documents the detected issues and their resolutions.

Re-execution under the revised plan Π' yields a new reasoning trajectory and a definitive decision $y \in \{\text{T, F, U}\}$. Thus, the entire feedback-driven process can be expressed concisely as:

$$\text{FD-Replan} : (\mathcal{H}, \Pi, \tau, \hat{y}) \mapsto (\Pi', \tau', y), \quad (14)$$

where τ' is the updated execution log under Π' , and y is the final three-valued answer consistent with the revised dependencies.

► **Input:**
Please replan: 1) identify the deficiencies of the matrix-based plan and regenerate a revised plan; 2) subsequently re-execute the new plan.
<Structured symbolic representation> ($\mathcal{H} = \{P_c, Q_c\}$)
<Matrix-based Plan> (Π)
<Execution log> (τ)
<Provisional answer> ($\hat{y}$)
► **Output:**
{
 "Revised plan (Π')": {
 "Corrected plan": {
 "1": { "content": "Establish all initial facts from the premises." },
 (... More plans ..)
 },
 "Matrix": {
 [0,1,0,0],
 ...,
 [0,0,0,0]
 }
 }

```
}
}
"Updated Execution log" ( $\tau'$ ): "Establish Initial Facts from Premises..."
"Final answer ( $y$ )": "T (True)"
}
```

4 Experiments

4.1 Experimental Setup

Dataset. We evaluate on the same set of five logical-reasoning datasets: AR-LSAT [36], LogicalDeduction [24], FOLIO [7], ProofWriter [25], and PrOntoQA [22]. These datasets span multiple symbolic representations and problem configurations, allowing us to test applicability and robustness across heterogeneous settings. PrOntoQA, ProofWriter, and FOLIO follow a first-order logic paradigm. LogicalDeduction and AR-LSAT are predominantly constraint-based. Together they cover different reasoning depths and choice cardinalities (e.g., T/F/U vs. multiple options). To avoid bias from differences in data splits, we adopt the same test subsets as Logic-LM and SymbCoT. During evaluation, any output with formatting non-compliance or missing fields is counted as incorrect. The detailed prompt templates are presented in Appendix A.1.

LLMs. To comprehensively assess the effectiveness of the proposed method, we conduct comparative experiments on both closed- and open-source large language models. The closed-source models are GPT-4o (2024-11-20) and GPT-4o-mini (2024-07-18). The open-source models are Qwen2.5-72B, DeepSeek-V3, and Kimi-K2. To ensure a fair comparison and attribute performance differences primarily to the method rather than to prompt engineering, we evaluate each method on each model using the same prompt. Task instructions, exemplars, and output formatting are held fixed. Unless otherwise specified, all other inference settings are likewise kept constant.

Baselines. We adopt representative methods as baselines: Logic-LM [17], SymbCoT [32], and Aristotle [31]. We also include two generic prompting styles—standard prompting and chain-of-thought (CoT) prompting that leverages the in-context learning capability of base LLMs. We conduct a unified evaluation on the five datasets described above. However, to ensure strictly fair comparability, the public implementation of Aristotle provides neither results nor reproducible configurations for FOLIO, AR-LSAT, or LogicalDeduction. Accordingly, we report their numbers only on PrOntoQA and ProofWriter; the remaining three entries are marked with “—”.

4.2 Main Result

4.2.1 Overall Performance. As shown in Tables 1 and 2, MatrixCoT attains the best—or tied best—accuracy on both closed-source and open-source model families. Its variance across models and datasets is small. This indicates strong robustness and transferability. To ensure comparability, all methods use identical prompt templates and output constraints on each model. Under this setting, our method still leads. The gains therefore stem primarily from the method itself rather than from tailoring to a particular model or prompt.

4.2.2 Method Differences and Empirical Observations. We observe that CoT achieves standout single-point results on certain models

Method	GPT-4o-mini					GPT-4o				
	PrOntoQA	ProofWriter	FOLIO	LogicalDeduction	AR-LSAT	PrOntoQA	ProofWriter	FOLIO	LogicalDeduction	AR-LSAT
Std	56.00	35.00	65.20	57.00	22.94	77.20	53.33	64.71	71.64	29.00
CoT	92.60	48.67	67.65	79.00	23.38	99.60	66.17	73.53	84.67	28.57
Logic-LM	80.80	44.50	51.47	85.00	20.78	80.60	79.17	67.16	91.00	30.74
SymbCoT	65.80	62.67	66.67	77.30	23.81	99.60	76.50	74.02	85.67	38.53
Aristotle	69.00	51.85	—	—	—	75.95	74.12	—	—	—
Ours	98.00	71.50	71.57	86.33	37.23	99.80	90.33	79.41	95.00	41.13

Table 1: Accuracy of six methods across five reasoning datasets on GPT-4o-mini and GPT-4o. The second best score is underlined and the best is **bold. “—” = N/A.**

Method	Qwen2.5-72B					DeepSeek-V3					Kimi-K2				
	PrOnto-QA	Proof-Writer	FOLIO	Logical-Deduction	AR-LSAT	PrOnto-QA	Proof-Writer	FOLIO	Logical-Deduction	AR-LSAT	PrOnto-QA	Proof-Writer	FOLIO	Logical-Deduction	AR-LSAT
Std	67.00	49.00	67.16	66.33	34.63	72.20	58.00	63.73	67.00	29.44	53.20	48.67	65.69	70.00	36.36
CoT	98.40	66.67	67.65	87.67	30.74	94.40	62.50	72.06	91.33	48.05	94.80	68.17	71.08	93.00	72.29
Logic-LM	72.80	67.17	64.71	87.00	29.87	78.20	75.83	67.16	93.67	37.66	96.00	78.83	69.61	97.00	42.86
SymbCoT	86.60	82.00	37.75	87.33	24.29	93.40	85.67	79.41	96.00	71.43	99.40	84.50	79.90	87.33	72.73
Aristotle	25.60	82.58	—	—	—	73.60	59.42	—	—	—	88.20	58.05	—	—	—
Ours	99.20	88.33	79.70	88.67	50.22	99.80	92.50	81.77	99.67	74.89	99.40	88.17	80.88	97.33	79.65

Table 2: Accuracy of six methods across five reasoning datasets on three open models. The second best score is underlined and the best is **bold. “—” = N/A.**

and datasets. We conjecture that this reflects extensive training exposure to chain-style narratives, which confers a format-alignment advantage on some tasks. However, this advantage fluctuates considerably across models and datasets. By contrast, although SymbCoT is also an LLM-driven framework, its textual planning and end-stage verification are sensitive to wording and sampling. It lacks machine-checkable cross-step dependencies and a process-level error-correction channel. As a result, it shows higher variance across models and is more prone to broken chains and omissions on long-chain or multi-branch cases. MatrixCoT employs a dependency-matrix-based plan that removes the uncertainty of “guessing dependencies” at execution time. It then absorbs issues exposed at runtime as structural repairs via minimal dependency edits driven by execution feedback. A unified structured output further reduces the risk of errors being counted due to format non-compliance. Consequently, it delivers a more stable and auditable performance profile across diverse base models.

4.3 Ablation Study

As shown in Table 3, we conduct an ablation study of MatrixCoT. We remove three modules in turn: matrix-based planning (wo MP), structured reasoning data management (wo SRM), and execution-feedback replanning (wo EFR). The full configuration without any ablation consistently delivers the best performance. Removing MP lowers average accuracy by about 9 percentage points. Removing SRM or EFR each yields a drop of roughly 5–6 percentage points. The gain from EFR is more pronounced on long-chain or multi-branch tasks. On shallower reasoning tasks, sensitivity to EFR is lower, but MP and SRM still provide substantial benefits.

We hypothesize three main reasons for these improvements. 1) SRM converts free text into typed, indexable symbolic carriers. It explicitly constrains predicate signatures and referential links, curbing semantic drift and supplying verifiable anchors. 2) MP replaces

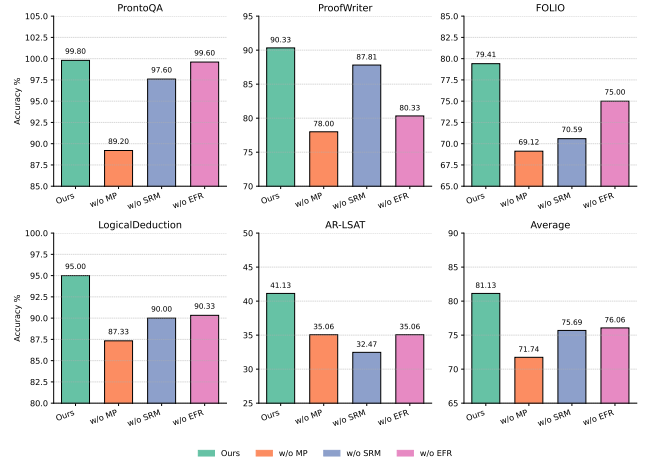


Figure 3: Ablation study. MP: Matrix-based plan; SRM: Structured data management; EFR: Feedback-Driven Replanner.

a linear narrative with a precedence–successor dependency matrix. It turns “dependency discovery” into “dependency execution.” Through topological scheduling and transitive reduction, it preserves the minimal sufficient dependencies. This reduces broken chains, premature use, and omissions, and it facilitates cross-sample alignment and reuse. 3) EFR transforms execution traces and candidate answers into local minimal edits to the matrix and then re-executes. It feeds issues exposed at runtime back as provable repairs at the structural layer. Together, these three components elevate intermediate evidence from implicit narrative to computable, auditable artifacts, enabling an LLM-driven framework to achieve

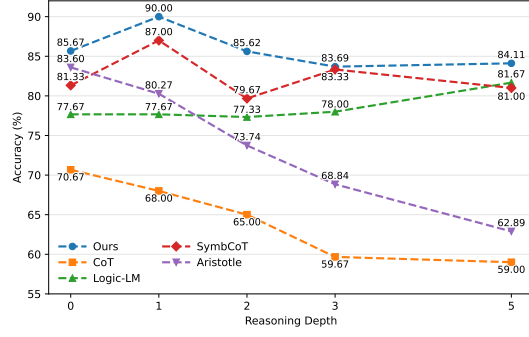


Figure 4: Accuracy on ProofWriter as reasoning depth increases.

greater robustness, controllability, and consistency without external solvers.

4.4 Analysis and Discussion

Reasoning Depth. As presented in Fig. 4, we conduct a stratified evaluation of ProofWriter on DeepSeek-V3 across reasoning depths (from 0 to 5). MatrixCoT achieves the highest accuracy at every depth. Its accuracy declines only slightly as depth increases. Overall, it remains stable at roughly 84%–90%. By contrast, the baselines drop sharply with increasing depth. These results suggest that a dependency-based, matrix-structured plan provides explicit precedence constraints that guide execution and scheduling. It mitigates error accumulation in long-chain reasoning. Even at greater depths, it sustains robust performance.

Cross-Model Mean Performance. As can be seen in Fig. 5, on GPT-4o-mini, our mean accuracy is 72.93, ranking first. After upgrading to GPT-4o, it remains highest at 81.13. More importantly, the gain is only +8.21 percentage points—the smallest among methods covering all five datasets (Std +11.95; CoT +8.25; Logic-LM +13.22; SymbCoT +15.61). This indicates weaker reliance on increasing model size and smooth scaling on stronger models. The baselines achieve larger gains after the upgrade, yet their overall performance still does not exceed ours. This further shows greater stability and robustness across model capacities. Note that Aristotle covers only two datasets; its mean and gain are not directly comparable and are therefore excluded from the statistics.

Error Analysis. Reliable symbolic reasoning depends on three capabilities: 1) Information acquisition and structuring with target-relevant content; 2) Path selection and plan orchestration by explicitly modeling precedence–successor constraints and maintaining global consistency; and 3) Executing tasks with constraints and accuracy by performing semantically correct derivations according to the specified plan. MatrixCoT strengthens the first two via structured reasoning data management and a matrix-based plan. Structured reasoning data management provides a typed, alignable symbolic base. The matrix plan fixes executable precedence relations through a dependency matrix A . Together, they make “finding the right information and laying out the correct path” a high-probability event. On this basis, execution is typically smoother and yields robust gains. For models with weak instruction-following

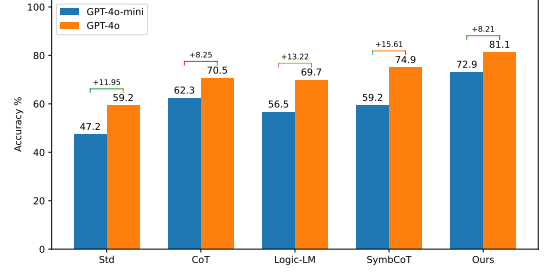


Figure 5: Improvement from GPT-4o-mini to GPT-4o.

ability, execution can still fail. It occurs even when the plan is logically complete and its dependencies are explicit. The failures often stem from an inadequate grasp of the dependency constraints. Thus, MatrixCoT reduces reliance on free-text planning and ad hoc search, but it does not substitute for the base model’s core execution ability. As a model’s basic comprehension and compliance improve with scale, it exploits the matrix plan more effectively, and the gains become more stable.

5 Conclusion

In this paper, we introduce MatrixCoT, a structured CoT framework that integrates a matrix-based method and a feedback-driven replanning mechanism. Specifically, we enforce a unified structured output format and normalize solution steps as typed symbolic representations, while explicitly encoding cross-step constraints via a dependency matrix. We also derive a verifiable, executable step-wise plan, and introduce a feedback-driven replanning mechanism, which can repair broken chains and correct deviations. The entire procedure operates without any external solver, yet improves robustness, interpretability, and controllability. Extensive experiments are conducted on five standard benchmarks and five LLMs (both closed- and open-source), demonstrating stable and competitive performance across all settings.

Acknowledgments

This work was supported in part by the Joint Funds of the National Natural Science Foundation of China under Grant U25A20436; by the Guangxi Key Research & Development Program (FN2504240036, 2025FN96441087); by the National Natural Science Foundation of China (NSFC) (62372047, 62302049); by the Natural Science Foundation of Guangdong Province (2024A1515011323); by the Supplemental Funds for Major Scientific Research Projects of Beijing Normal University, Zhuhai (ZHPT2023002); by the Fundamental Research Funds for the Central Universities, Ministry of Education Industry-University Cooperation Collaborative Education Project (240904497110437); and by the Higher Education Research Topics of the Guangdong Association of Higher Education in the 14th Five-Year Plan (24GYB207). We acknowledge the support of the Interdisciplinary Intelligence Super Computer Center of Beijing Normal University at Zhuhai.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 17682–17690.
- [3] Bohan Chen and Andrea L Bertozzi. 2023. AutoKG: Efficient automated knowledge graph generation for language models. In *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 3117–3126.
- [4] Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [5] Simon Frieder, Jonas Bayer, Katherine M Collins, Julius Berner, Jacob Loader, András Juhász, Fabian Ruehle, Sean Welleck, Gabriel Poesia, Ryan-Rhys Griffiths, et al. 2024. Data for mathematical copilots: Better ways of presenting proofs for machine learning. *arXiv preprint arXiv:2412.15184* (2024).
- [6] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. In *International Conference on Machine Learning*. PMLR, 10764–10799.
- [7] Simeng Han, Hailey Schoolkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, et al. 2024. Folio: Natural language reasoning with first-order logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 22017–22031.
- [8] Shashank Kirtania, Priyanshu Gupta, and Arjun Radhakrishna. 2024. LOGIC-LM++: Multi-Step Refinement for Symbolic Formulations. In *Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations (@ ACL 2024)*. 56–63.
- [9] Ivan Leo. 2024. Bad Schemas Could Break Your LLM Structured Outputs. <https://python.useinstructor.com/blog/2024/09/26/bad-schemas-could-break-your-llm-structured-outputs/>. Instructor Blog. Accessed 2025-10-08.
- [10] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [11] Ben Liu, Jihai Zhang, Fangquan Lin, Cheng Yang, Min Peng, and Wotao Yin. 2025. Symagent: A neural-symbolic self-learning agent framework for complex reasoning over knowledge graphs. In *Proceedings of the ACM on Web Conference 2025*. 98–108.
- [12] Tongxuan Liu, Wenjiang Xu, Weizhe Huang, Yuting Zeng, Jiaxing Wang, Xingyu Wang, Hailong Yang, and Jing Li. 2025. Logic-of-Thought: Injecting Logic into Contexts for Full Reasoning in Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 10168–10185.
- [13] Man Luo, Shrinidhi Kumbhar, Mihir Parmar, Neeraj Varshney, Pratyay Banerjee, Somak Aditya, Chitta Baral, et al. 2023. Towards logigluue: A brief survey and a benchmark for analyzing logical reasoning capabilities of language models. *arXiv preprint arXiv:2310.00836* (2023).
- [14] Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. 2024. Enhancing reasoning capabilities of llms via principled synthetic logic corpus. *Advances in Neural Information Processing Systems* 37 (2024), 73572–73604.
- [15] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332* (2021).
- [16] Theo Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua Tenenbaum, and Roger Levy. 2023. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 5153–5176.
- [17] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 3806–3824.
- [18] Yudai Pan, Jiajie Hong, Tianzhe Zhao, Lingyun Song, Jun Liu, and Xuequn Shang. 2025. Logic-Aware Knowledge Graph Reasoning for Structural Sparsity under Large Language Model Supervision. In *Proceedings of the ACM on Web Conference 2025*. 4531–4542.
- [19] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. 2024. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems* 37 (2024), 30811–30849.
- [20] Jaechul Roh, Varun Gandhi, Shivani Anilkumar, and Arin Garg. 2025. Chain-of-Code Collapse: Reasoning Failures in LLMs via Adversarial Prompting in Code Generation. *arXiv preprint arXiv 2506* (2025).
- [21] Hyun Ryu, Gyeongman Kim, Hyemin S. Lee, and Eunho Yang. 2025. Divide and Translate: Compositional First-Order Logic Translation and Verification for Complex Logical Reasoning. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net.
- [22] Abulhair Saparov and He He. 2023. Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023*. OpenReview.net.
- [23] Parshin Shojade, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. 2025. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. *arXiv preprint arXiv:2506.06941* (2025).
- [24] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Shueb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adri Garriga-Alonso, et al. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on machine learning research* (2023).
- [25] Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. 3621–3634.
- [26] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. Let me speak freely? a study on the impact of format restrictions on performance of large language models. *arXiv preprint arXiv:2408.02442* (2024).
- [27] Zengzhi Wang, Xuefeng Li, Rui Xia, and Pengfei Liu. 2024. Mathpile: A billion-token-scale pretraining corpus for math. *Advances in Neural Information Processing Systems* 37 (2024), 25426–25468.
- [28] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [29] Tong Wu, Chong Xiang, Jiachen T Wang, G Edward Suh, and Prateek Mittal. 2025. Effectively controlling reasoning models through thinking intervention. *arXiv preprint arXiv:2503.24370* (2025).
- [30] Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang, and Enhong Chen. 2024. Large language models for generative information extraction: A survey. *Frontiers of Computer Science* 18, 6 (2024), 186357.
- [31] Jundong Xu, Hao Fei, Meng Luo, Qian Liu, Liangming Pan, William Yang Wang, Preslav Nakov, Mong-Li Lee, and Wynne Hsu. 2025. Aristotle: Mastering logical reasoning with a logic-complete decompose-search-resolve framework. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3052–3075.
- [32] Jundong Xu, Hao Fei, Liangming Pan, Qian Liu, Mong-Li Lee, and Wynne Hsu. 2024. Faithful Logical Reasoning via Symbolic Chain-of-Thought. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13326–13365.
- [33] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [34] Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. Satlm: Satisfiability-aided language models using declarative prompting. *Advances in Neural Information Processing Systems* 36 (2023), 45548–45580.
- [35] Ori Yoran, Samuel Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. AssistantBench: Can Web Agents Solve Realistic and Time-Consuming Tasks?. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 8938–8968.
- [36] Wanjuan Zhong, Siyuan Wang, Duyu Tang, Zenan Xu, Daya Guo, Yining Chen, Jiahai Wang, Jian Yin, Ming Zhou, and Nan Duan. 2022. Analytical reasoning of text. In *Findings of the Association for Computational Linguistics: NAACL 2022*. 2306–2319.

A Appendix

A.1 Prompt Templates

We present a brief description of the prompt setup used on the ProofWriter dataset. A total of six prompts are employed, organized into four modules: Translator, Matrix-based Planner, Solver, and Replanner (which includes Replanning and Re-Execution). These prompts together support the complete process from natural language propositions to automated reasoning and verification.

Prompt: Translator

You are a logical reasoning parser that converts natural language deduction problems into a structured, symbolic format based on first-order logic. Your goal is to extract entities, predicates (properties and relations), and translate all premises (axioms) and the question (query) into their logical forms. Return a JSON object with the following structure:

- **entities****: A list of all unique individuals mentioned (e.g., "bob", "charlie", "gary"). Use lowercase for consistency.
- **predicates****: A list of all unique properties (unary predicates) and relations (binary predicates).
 - "name": The predicate name (e.g., "IsCold", "Sees"). Use PascalCase.
 - "arity": The number of arguments the predicate takes (1 for properties, 2 for relations).
- **axioms****: A list of all premises from the context, translated into symbolic form. Each object should have:
 - "statement": The original or a closely paraphrased natural language sentence.
 - "symbol": The symbolic representation using the defined predicates and entities. Use "forall" for universal quantifiers.
- **query****: An object representing the statement to be evaluated, containing:
 - "statement": The natural language question.
 - "symbol": The symbolic representation of the query.
- **options****: The list of possible answers (e.g., True, False, Unknown).

Example
****Context****:
 Bob is cold. Bob is not nice. Bob is not quiet. Charlie is blue. Charlie is round. Gary is rough. Harry is cold. Rough things are blue. Red, quiet things are blue. All rough, blue things are nice. If something is quiet and round then it is not nice. If Harry is red then Harry is rough. If something is nice then it is round. If something is round then it is red. Red things are not quiet.

****Question****:
 Based on the above information, is the following statement true, false, or unknown? Gary is red.

Expected Output:

```

{
  "json":
  {
    "entities": ["bob", "charlie", "gary", "harry"],
    "predicates": [
      { "name": "IsCold", "arity": 1 }, { "name": "IsNice", "arity": 1 }, { "name": "IsQuiet", "arity": 1 }, { "name": "IsBlue", "arity": 1 },
      { "name": "IsRound", "arity": 1 }, { "name": "IsRough", "arity": 1 }, { "name": "IsRed", "arity": 1 } ],
    "axioms": [
      { "statement": "Bob is cold.", "symbol": "IsCold(bob)" }, { "statement": "Bob is not nice.", "symbol": "not(IsNice(bob))" },
      { "statement": "Bob is not quiet.", "symbol": "not(IsQuiet(bob))" }, { "statement": "Charlie is blue.", "symbol": "IsBlue(charlie)" },
      { "statement": "Charlie is round.", "symbol": "IsRound(charlie)" }, { "statement": "Gary is rough.", "symbol": "IsRough(gary)" },
      { "statement": "Harry is cold.", "symbol": "IsCold(harry)" }, { "statement": "Rough things are blue.", "symbol": "forall X: (IsRough(X) -> IsBlue(X))" },
      { "statement": "Red, quiet things are blue.", "symbol": "forall X: ((IsRed(X) and IsQuiet(X)) -> IsBlue(X))" },
      { "statement": "All rough, blue things are nice.", "symbol": "forall X: ((IsRough(X) and IsBlue(X)) -> IsNice(X))" },
      { "statement": "If something is quiet and round then it is not nice.", "symbol": "forall X: ((IsQuiet(X) and IsRound(X)) -> not(IsNice(X)))" },
      { "statement": "If Harry is red then Harry is rough.", "symbol": "IsRed(harry) -> IsRough(harry)" },
      { "statement": "If something is nice then it is round.", "symbol": "forall X: (IsNice(X) -> IsRound(X))" },
      { "statement": "If something is round then it is red.", "symbol": "forall X: (IsRound(X) -> IsRed(X))" },
      { "statement": "Red things are not quiet.", "symbol": "forall X: (IsRed(X) -> not(IsQuiet(X)))" },
    ],
    "query": { "statement": "Gary is red.", "symbol": "IsRed(gary)" } }
  },
  ...
}

```

Now do the same for the following input:
 Context: [[CONTEXT]] Question: [[QUESTION]] Options: [[CHOICES]]
 Only return the JSON object.

Prompt: Matrix-based Planner

You are given a structured **logical deduction problem**, represented in first-order logic. Your task is to generate a **symbolic reasoning plan** to determine if a given query is **True**, **False**, or **Unknown** based on a set of axioms. This plan should be represented as:

- * A dictionary of step-nodes with textual explanations.
- * A dependency **adjacency matrix** indicating which steps depend on which.

Output Structure

You must return a JSON object with **two keys**:

- "plan": A dictionary mapping step IDs ("1", "2", etc.) to reasoning step objects:
 "" json { "key": "<short description of reasoning at this step>" } ""
- "matrix": A binary square matrix where "matrix[i][j] = 1" indicates that step "i+1" is a direct **prerequisite** of step "j+1".

Reasoning Phases

Your plan should follow these logical phases, each with one or more nodes:

1. **Knowledge Base Setup**

* Parse entities and predicates from input. * Load all axioms (facts and rules) into a formal knowledge base.

2. **Deductive Reasoning**

- * Initialize inference process (e.g., forward chaining).
- * Apply inference rules such as: * Universal Instantiation * Modus Ponens * Conjunction Elimination
- * Derive new facts until saturation (deductive closure).

3. **Query Evaluation**

- * State the query being tested. * Check whether the query is derivable from facts.
- * Check whether the negation of the query is derivable.

4. **Final Judgment**

* If query derivable → True * If negation derivable → False * Otherwise → Unknown

Matrix-Based Reasoning Plan Output

```

{
  "json":
  {
    "plan": {
      "1": { "key": "Parse entities and predicates from input." },
      "2": { "key": "Load all axioms (facts and rules) into a knowledge base." },
      "3": { "key": "Start forward-chaining inference procedure." },
      "4": { "key": "Apply inference rules (e.g., Modus Ponens, Universal Instantiation) to derive all possible facts." },
      "5": { "key": "Identify the query: IsRed(gary)." },
      "6": { "key": "Check if IsRed(gary) is present in the derived facts." },
      "7": { "key": "Check if not(IsRed(gary)) is derivable from the knowledge base." },
      "8": { "key": "Make final judgment based on presence or absence of IsRed(gary) or its negation." }
    },
    "matrix": [
      [0, 1, 0, 0, 0, 0, 0, 0], // 1 → 2
      [0, 0, 1, 0, 0, 0, 0, 0], // 2 → 3
      [0, 0, 0, 1, 0, 0, 0, 0], // 3 → 4
      [0, 0, 0, 0, 1, 1, 0, 0], // 4 → 5,6,7
      [0, 0, 0, 0, 0, 0, 1, 0], // 5 → 8
      [0, 0, 0, 0, 0, 0, 0, 1], // 6 → 8
      [0, 0, 0, 0, 0, 0, 1, 0], // 7 → 8
      [0, 0, 0, 0, 0, 0, 0, 0] // 8 → done
    ]
  },
  ...
}

```

INPUT
 [[CONTEXT]]

Prompt: Solver

You are a logical deduction engine executing a **symbolic reasoning plan** over first-order logic. You are given:

- Structured Logical Problem**:
 - A list of **entities** (e.g., "gary", "bob").
 - A list of **predicates** (e.g., "IsRed", "IsNice").
 - A list of **axioms** (first-order logic facts and rules).
 - A **query** (e.g., "IsRed(gary)").
 - A list of **options** ("A", "B", "C" for True, False, Unknown).
- Reasoning Plan**, expressed as:
 - A dictionary "plan" mapping node IDs ("1", "2", etc.) to logical steps.
 - An adjacency "matrix" where "matrix[i][j] = 1" means step "i+1" is a prerequisite of step "j+1".

Your Task:

- Execute the Plan in Topological Order**:
 - Respect all dependencies defined in the adjacency matrix. Only execute a step when its prerequisites have been completed.
- Build the Knowledge Base**:
 - Parse and store all facts and rules from the "axioms".
- Run Forward-Chaining Inference**:
 - Apply universal rules to current known facts.
 - Use inference rules like:
 - Modus Ponens**: if "A $\rightarrow$ B" and "A", infer "B".
 - Conjunction Elimination**: from "A $\wedge$ B", infer "A", "B".
 - Universal Instantiation**: apply " $\forall X$ " to constants in "entities".
- Trace All Deductions**:
 - Explicitly show how each new fact is inferred from which rule + which known facts. You must continue inference until no new facts can be derived.
- Evaluate the Query**:
 - If the query is in the derived facts, it's **True** (Option A).
 - If the **negation** of the query is derivable, it's **False** (Option B).
 - If neither, the answer is **Unknown** (Option C).
- Output Final Answer**:
 - Output only the letter corresponding to your conclusion.

Example Input Problem

```

{
  "entities": ["gary"],
  "predicates": [
    { "name": "IsRough", "arity": 1 },
    { "name": "IsBlue", "arity": 1 },
    { "name": "IsNice", "arity": 1 },
    { "name": "IsRound", "arity": 1 },
    { "name": "IsRed", "arity": 1 }
  ],
  "axioms": [
    { "logic": "IsRough(gary)", "type": "fact" },
    { "logic": "forall X: (IsRough(X) -> IsBlue(X))", "type": "rule" },
    { "logic": "forall X: ((IsRough(X) and IsBlue(X)) -> IsNice(X))", "type": "rule" },
    { "logic": "forall X: (IsNice(X) -> IsRound(X))", "type": "rule" },
    { "logic": "forall X: (IsRound(X) -> IsRed(X))", "type": "rule" }
  ],
  "query": { "logic": "IsRed(gary)" },
  "options": { "A": "True", "B": "False", "C": "Unknown" }
}

```

Reasoning Plan

```

{
  "plan": {
    "1": { "key": "Parse and load all axioms into the knowledge base." },
    "2": { "key": "Begin inference: apply forward chaining to derive new facts from axioms." },
    "3": { "key": "Track all logical implications using universal instantiation + modus ponens." },
    "4": { "key": "Evaluate whether IsRed(gary) is derivable from inferred facts." },
    "5": { "key": "Check whether not(IsRed(gary)) is derivable." },
    "6": { "key": "Final judgment: A if proven, B if disproven, else C." }
  },
  "matrix": [
    [0, 1, 0, 0, 0, 0], // Step 1 -> Step 2
    [0, 0, 1, 1, 1, 0], // Step 2 -> Step 3,4,5
    [0, 0, 0, 0, 0, 1], // Step 3 -> Step 6
    [0, 0, 0, 0, 0, 1], // Step 4 -> Step 6
    [0, 0, 0, 0, 0, 1], // Step 5 -> Step 6
    [0, 0, 0, 0, 0, 0] // Step 6
  ]
}

```

Step-by-Step Reasoning

Step 1 – Load all axioms:
 * Initial Fact: "IsRough(gary)" * Rules: 1. $\forall X: \text{IsRough}(X) \rightarrow \text{IsBlue}(X)$ 2. $\forall X: \text{IsRough}(X) \wedge \text{IsBlue}(X) \rightarrow \text{IsNice}(X)$ 3. $\forall X: \text{IsNice}(X) \rightarrow \text{IsRound}(X)$ 4. $\forall X: \text{IsRound}(X) \rightarrow \text{IsRed}(X)$

Step 2-3 – Inference via Forward Chaining:
 1. From "IsRough(gary)" and Rule 1: * Infer "IsBlue(gary)" 2. From "IsRough(gary)" $\wedge$ "IsBlue(gary)" and Rule 2: * Infer "IsNice(gary)"
 3. From "IsNice(gary)" and Rule 3: * Infer "IsRound(gary)" 4. From "IsRound(gary)" and Rule 4: * Infer "IsRed(gary)"
 * Derived Facts: "IsRough(gary), IsBlue(gary), IsNice(gary), IsRound(gary), IsRed(gary)"

Step 4 – Evaluate query: * Query = "IsRed(gary)" * Present in derived facts. **Step 5** – Evaluate negation: * "not(IsRed(gary))" is not derivable. **Step 6** – Final Conclusion Query is provably True $\Rightarrow$ Answer = "A"

Final Output

```

{
  "final_answer": "A"
}

```

Now solve the following logical problem by executing the reasoning plan:
 Context: [[CONTEXT]] Reasoning Plan: [[PLAN]]

Prompt: Replanning

You are an expert **Logician** and **AI Strategist**. Your task is to analyze a failed logical deduction, diagnose the root cause of the error, and create a **superior reasoning plan**. The new plan must be clear, efficient, and—most importantly—**appropriate** for the specific problem structure.

Core Principles

The Principle of Simplicity (Occam's Razor): Always prefer the simplest effective strategy. A direct, step-by-step forward-chaining proof is the **default** and most common solution.

Do not resort to more complex strategies like "Proof by Contradiction" or "Case Analysis" unless you can demonstrate that a direct proof is impossible.

The Principle of Robustness: Your new plan must be explicit and designed to guide the next AI away from common logical fallacies. Your final evaluation step must contain a clear warning about how to handle uncertainty correctly.

Instructions:

- Analyze the Failure**: First, meticulously analyze the "Flawed Execution". Compare its reasoning steps against the "CONTEXT" (facts and rules) to pinpoint the exact logical error. Did the execution miss a rule, apply a rule incorrectly, or halt its reasoning prematurely?
- Diagnose the Strategic Flaw (with a Simplicity Bias)**:
 - Default to Direct Proof**: Start with the assumption that the failure was a simple
 - Failure of Thoroughness**: Is it possible to find the correct answer by just applying the rules more exhaustively in a forward chain? This is almost always the correct diagnosis.
 - Consider Complex Strategies as a Last Resort**: Only if you consider "Proof by Contradiction" or "Case Analysis" should you consider them.
- Formulate an Appropriate, Corrective Plan**: Based on your diagnosis, create a new plan using the **correct** strategy.
 - For Direct Proofs (Default)**: Your plan should be a simple, exhaustive forward-chaining process. Consolidate fact-gathering and create a single, powerful step like, "Systematically apply all rules to all known facts, repeating until deductive closure is reached."
 - Embed Safeguards**: Your final evaluation step **must** explicitly warn against the "Closed-World Assumption." See the example below for the correct phrasing. This is crucial for preventing future errors.
- Produce the New Plan**: Your final output must be a single JSON object containing your justification and the complete, new plan.
 - Dependency representation requirement (matrix-based)**: Encode step dependencies with an adjacency **matrix** instead of per-step lists. Provide:
 - "plan": step dictionary where each step has only "key".
 - "matrix": an "N×N" binary matrix where "matrix[i][j] = 1" means **step i+1** is a prerequisite of step **j+1**.

JSON Output Format:

```

{
  "justification_for_changes": "<Briefly explain the flaw and the chosen strategy, emphasizing simplicity. Example: 'The original execution halted prematurely and missed a key deduction. The problem has a clear direct proof. My new plan enforces a simple, exhaustive forward-chaining strategy and explicitly forbids assuming falsity from a lack of proof.'>",
  "corrected_plan": {
    "plan": {
      "1": { "key": "Establish all initial facts from the premises." },
      "2": { "key": "Systematically apply all rules to all known facts in a forward chain, repeating until no new facts can be derived (reaching deductive closure)." },
      "3": { "key": "After reaching deductive closure, check if the query or its negation is present in the final set of derived facts." },
      "4": { "key": "Final Judgment. CRITICAL REMINDER: If neither the query nor its negation was proven in the previous step, the answer MUST be 'Uncertain'. Do not assume a statement is false simply because it was not proven true (avoid the Closed-World Assumption). Conclude True, False, or Uncertain based on this strict rule." }
    },
    "matrix": [
      [0,1,0,0], // 1 -> 2
      [0,0,1,0], // 2 -> 3
      [0,0,0,1], // 3 -> 4
      [0,0,0,0] // 4 terminal
    ]
  }
}

```

 *Context Data: [[CONTEXT_PLACEHOLDER]] *Original Plan Data: [[PLAN_PLACEHOLDER]]
 *Flawed Execution Data: [[EXECUTION_PLACEHOLDER]]

Prompt: Re-Execution

You are a symbolic logic reasoner. Your goal is to solve a first-order logic problem by executing a predefined reasoning plan with utmost logical rigor.

You are given:

* A **structured logic problem**, including "predicates", "entities", "axioms", a "query", and "options".

* A **reasoning plan package** containing:

* **"justification_for_changes"**: a short rationale explaining why the plan was corrected and what strategic bias to adopt (e.g., prefer direct forward-chaining over unnecessary case splits; avoid Closed-World Assumption).

* **"corrected_plan"**:

* "plan": a dictionary of reasoning steps, each with a "key" description.

* "matrix": an $N \times N$ binary adjacency matrix where $matrix[i][j] = 1$ means step $i+1$ is a prerequisite of step $j+1$ (this replaces per-step dependency lists and defines the execution order).

Your Task and Core Principles

CRITICAL REASONING PRINCIPLE: Operate under the Open-World Assumption (OWA).

* A statement's truth value is **Uncertain** by default.

* Conclude **True** or **False** only if the statement (or its negation) is a **direct, provable consequence** of the axioms via sound inference.

**Do not assume a statement is false merely because it was not proven true (avoid the Closed-World Assumption).

Your Task:

1. **Execute the Matrix-Based Plan**

Read "justification_for_changes" first to adopt the intended strategy (e.g., **direct forward-chaining** unless the justification proves branching is necessary).

Then execute the steps **strictly** in the partial order implied by "matrix". If a step in the plan is illogical or inefficient, explicitly call it out and proceed with the correct, logically sound action consistent with the justification.

2. **Perform Exhaustive Forward Chaining**

Systematically apply all universally quantified rules to all known facts, repeatedly, until **deductive closure** (no new facts can be derived).

3. **Evaluate the Query (Strict Procedure)**

After reaching closure:

* **Step 1** (Check for TRUE): If the query is in the derived facts, the answer is **True**.

* **Step 2** (Check for FALSE): If the **negation** of the query is in the derived facts, the answer is **False**.

* **Step 3** (Default to UNCERTAIN): If neither appears, the answer **must** be **Uncertain**.

4. **Final Judgment**

Output your result as a single option letter in the strict JSON format.

Example (FOL — Direct Forward-Chaining with Matrix Plan)

Context (complete):

'''json

{

 "entities": ["cat", "squirrel", "cow", "rabbit"],

 "predicates": { { "name": "Eats", "arity": 2 }, { "name": "Sees", "arity": 2 }, { "name": "IsRound", "arity": 1 }, { "name": "IsCold", "arity": 1 },

 { "name": "Needs", "arity": 2 }, { "name": "IsGreen", "arity": 1 }, { "name": "IsRough", "arity": 1 }, { "name": "IsKind", "arity": 1 } },

 "axioms": [{ "statement": "The cat eats the squirrel.", "logic": "Eats(cat, squirrel)" }, { "statement": "The cat sees the squirrel.", "logic": "Sees(cat, squirrel)" }, { "statement": "The cow eats the squirrel.", "logic": "Eats(cow, squirrel)" },

 { "statement": "The cow sees the cat.", "logic": "Sees(cow, cat)" }, { "statement": "The rabbit is round.", "logic": "IsRound(rabbit)" }, { "statement": "The rabbit sees the cat.", "logic": "Sees(rabbit, cat)" },

 { "statement": "The squirrel eats the rabbit.", "logic": "Eats(squirrel, rabbit)" }, { "statement": "The squirrel is cold.", "logic": "IsCold(squirrel)" },

 { "statement": "The squirrel needs the rabbit.", "logic": "Needs(squirrel, rabbit)" }, { "statement": "The squirrel sees the cat.", "logic": "Sees(squirrel, cat)" },

 { "statement": "If someone sees the cat and they are not green then they see the cow.", "logic": "forall X: (Sees(X, cat) and not(IsGreen(X)) -> Sees(X, cow))" },

 { "statement": "If the rabbit is kind and the rabbit sees the squirrel needs the rabbit.", "logic": "forall X: ((IsKind(rabbit) and Sees(rabbit, squirrel)) -> Needs(squirrel, rabbit))" },

 { "statement": "Rough people are cold.", "logic": "forall X: (IsRough(X) -> IsCold(X))" },

 { "statement": "If someone sees the rabbit then they are not round.", "logic": "forall X: (Sees(X, rabbit) -> not(IsRound(X)))" },

 { "statement": "If someone sees the squirrel and they are not green then they need the squirrel.", "logic": "forall X: (Sees(X, squirrel) and not(IsGreen(X)) -> Needs(X, squirrel))" },

 { "statement": "If someone eats the cow then they see the rabbit.", "logic": "forall X: (Eats(X, cow) -> Sees(X, rabbit))" },

 { "statement": "If someone eats the squirrel then they are rough.", "logic": "forall X: (Eats(X, squirrel) -> IsRough(X))" },

 { "statement": "If someone is cold then they eat the cow.", "logic": "forall X: (IsCold(X) -> Eats(X, cow))" },

 "query": { "statement": "The cat is not round.", "logic": "not(IsRound(cat))" },

 "options": ["A) True", "B) False", "C) Unknown"]

 }

'''

Reasoning Plan Package (matrix-based):

'''json

{ **"justification_for_changes"**: "The original execution halted prematurely and relied on case analysis, which complicated the reasoning unnecessarily. The problem has a clear direct proof path. My new plan enforces a simple,

exhaustive forward-chaining strategy that systematically applies all premises to derive conclusions about Bonnie, while explicitly forbidding the assumption of falsity from a lack of proof."

"corrected_plan": { "plan": { "1": { "key": "Establish all initial facts from the premises." },

 "2": { "key": "Systematically apply all rules to all known facts in a forward chain, repeating until no new facts can be derived (reaching deductive closure)."},

 "3": { "key": "After reaching deductive closure, check if the query or its negation is present in the final set of derived facts." },

 "4": { "key": "Final Judgment. CRITICAL REMINDER: If neither the query nor its negation was proven in the previous step, the answer **MUST** be 'Uncertain'. Do not assume a statement is false simply because it was not proven true (avoid the Closed-World Assumption). Conclude True, False, or Uncertain based on this strict rule." } },

 "matrix": [[0,1,0,0], [0,0,1,0], [0,0,0,1], [0,0,0,0]] }

'''

Note: The justification mentions "Bonnie" from a prior correction, but its strategic directive applies **generally**: adopt **direct forward-chaining**, avoid unnecessary case analysis, and **do not** assume falsity from non-derivation.

Step-by-Step Reasoning (executed in matrix order)

Step 1 — Establish initial facts

From axioms:

* $\text{Eats}(\text{cat}, \text{squirrel})$ * $\text{Sees}(\text{cat}, \text{squirrel})$ * $\text{Eats}(\text{cow}, \text{squirrel})$ * $\text{Sees}(\text{cow}, \text{cat})$ * $\text{IsRound}(\text{rabbit})$

* $\text{Sees}(\text{rabbit}, \text{cat})$ * $\text{Eats}(\text{squirrel}, \text{rabbit})$ * $\text{IsCold}(\text{squirrel})$ * $\text{Needs}(\text{squirrel}, \text{rabbit})$ * $\text{Sees}(\text{squirrel}, \text{cat})$

(plus all universal rules R1–R8 below)

Step 2 — Forward-chaining to deductive closure

Rules used (for reference):

* R3: $\text{IsRough}(X) \rightarrow \text{IsCold}(X)$ * R6: $\text{Eats}(X, \text{cow}) \rightarrow \text{Sees}(X, \text{rabbit})$ * R7: $\text{Eats}(X, \text{squirrel}) \rightarrow \text{IsRough}(X)$

* R8: $\text{IsCold}(X) \rightarrow \text{Eats}(X, \text{cow})$ * R4: $\text{Sees}(X, \text{rabbit}) \rightarrow \text{neg}(\text{IsRound}(X))$

Derivation for **cat**:

1. From $\text{Eats}(\text{cat}, \text{squirrel})$ and R7 $\Rightarrow \text{IsRough}(\text{cat})$. 2. From $\text{IsRough}(\text{cat})$ and R3 $\Rightarrow \text{IsCold}(\text{cat})$. 3. From $\text{IsCold}(\text{cat})$ and R8 $\Rightarrow \text{Eats}(\text{cat}, \text{cow})$.

4. From $\text{Eats}(\text{cat}, \text{cow})$ and R6 $\Rightarrow \text{Sees}(\text{cat}, \text{rabbit})$. 5. From $\text{Sees}(\text{cat}, \text{rabbit})$ and R4 $\Rightarrow \text{neg}(\text{IsRound}(\text{cat}))$.

(No rule derives $\text{IsRound}(\text{cat})$; the only positive "round" fact is for **rabbit**.)

Step 3 — Evaluate the query per plan

Query: $\text{neg}(\text{IsRound}(\text{cat}))$.

* Check TRUE: $\text{neg}(\text{IsRound}(\text{cat}))$ is derived (see 5).

* Therefore, the answer is **True**.

Step 4 — Final Judgment (with OWA safeguard)

Since we have a direct derivation of the query and no derivation of its negation, conclude **True**.

Final Output (Strict JSON):

'''json

{ "final_answer": "A" }

'''

Now, solve the following problem:

Context and Premises: "[CONTEXT]"

**Reasoning Plan Package (with "justification_for_changes", "corrected_plan.plan", and "corrected_plan.matrix"): "[PLAN]"